OpenMP Parallelization Strategies for Large-Scale CSV Data: Lessons from NYC 311

Bhuvana Komati Asha Vinodhini Vijayan Manasa Sadhu

March 9, 2026

Abstract

This report presents a performance analysis of serial vs. parallel processing on the NYC 311 Service Request dataset - a large-scale real-world dataset containing over 20 million records spanning 2010 to 2023. Our implementation loads 14 million records (∼1,062 MB in memory) from a 12.34 GB combined CSV file and benchmarks six query types across single-thread, multi-thread (OpenMP), and optimized configurations.

Our study reveals nuanced and often counterintuitive results: OpenMP parallelization improved performance for compute-bound queries with small result sets (complaint search: 1.27×, lat/lon box: 1.15×, average latitude: 1.36×), but significantly degraded performance for queries that copy large result sets or use `omp critical` sections for aggregation. Borough filter was 2.58× slower with 4 threads, and borough aggregation was 1.98× slower with 8 threads. These findings underscore that parallelization strategy, result size, and synchronization design all critically affect outcome.

1. Dataset: NYC 311 Service Requests (2010-2023)

Source	NYC Open Data - 311 Service Request dataset
Combined file	`311_combined.csv` (∼11 GB, 20.3 million total records)
Records loaded	14,000,000
Memory delta	∼1,062 MB (single thread)
Fields/record	44 columns (complaint type, borough, agency, lat/lon, dates, status, zip code, and more)

Data Structure: Each record is stored as a `ServiceRequest` struct containing primitive types (`uint64_t uniqueKey`, `uint32_t incidentZip`, `double latitude/longitude`, custom `DateTime` struct) and `std::string` fields for text columns. All records are held in a global `std::vector<ServiceRequest` (Array of Structs) loaded once at startup and queried in-place.

2. Overall Performance Comparison

The table below summarises all three implementations across all six queries, each averaged over 20 runs on 14 million records.

#	Query	Result Size	Single Avg (s)	Multi (6t) Avg (s)	Optimized (6t) Avg (s)
Load	Read 14M records	14,000,000	42.7317	46.2298	42.6789
1	Date range filter	484,799	3.2886	3.1940 (+2.9%)	**0.0081** (+99.8%)
2	Borough filter	4,002,729	87.0283	90.2450 (−3.7%)	**0.0374** (+99.96%)
3	Complaint keyword	165,106	4.8792	3.9199 (+19.7%)	**0.0890** (+98.2%)
4	Lat/Lon bounding box	12,673,163	3.3639	2.8907 (+14.1%)	**0.0501** (+98.5%)
5	Average latitude	scalar	2.8364	3.0662 (−8.1%)	**0.0018** (+99.9%)
6	Borough aggregation	8 zones	4.0679	3.8673 (+4.9%)	**0.1056** (+97.4%)

3. Single-Threaded Approach

The single-threaded implementation (`single_thread/main.cpp`) establishes the serial baseline against which all parallel results are measured.

3.1. Data Parsing and Loading

`loadData()` reads the CSV line-by-line using `std::getline()` with a 14M record cap. Each line is parsed by `parseCSVLine()`, a hand-written RFC-4180 CSV parser that handles quoted fields and escaped quotes. Fields are mapped into `ServiceRequest` types: dates to a custom `DateTime` struct, numeric strings to primitive types, and text columns to `std::string`.

3.2. Query Design

All six queries perform an $O(n)$ linear scan over the global `g_records` vector. Queries 1-4 return `vector<ServiceRequest>` result sets. Query 5 returns a scalar `double`. Query 6 returns a per-borough aggregation with total count and top complaint type.

- **Query 1 : Date range filter:** compare `createdDate` against a year boundary (2013). Matched 484,799 records.
- **Query 2 : Borough filter:** case-insensitive string match on the borough field. Matched 4,002,729 records for `BROOKLYN`.
- **Query 3: Complaint keyword:** substring search after lowercasing `complaintType`. Matched 165,106 records for *rodent*.
- **Query 4: Lat/Lon bounding box:** four floating-point comparisons per record. Matched 12,673,163 records.
- **Query 5 : Average latitude:** single-pass sum and divide over all 14M records.
- **Query 6 : Borough aggregation:** count records per borough and find the most common complaint type in each.

3.3. Timing Harness

Each query is executed 20 times using `std::chrono::high_resolution_clock`; the per-run average is reported.

Timing wraps only query logic - data loading is measured separately.

3.4. Execution Output

The single-thread run shows stable, predictable timings across all 20 repetitions with no variance from thread scheduling or lock contention. Query 2 (borough filter) is by far the most expensive at 87 s average, driven entirely by the cost of copying 4 million `std::string`-heavy structs into a result vector. Query 5 (average latitude) is the cheapest at 2.84 s - a tight arithmetic loop with no memory allocation. These results establish the true computational cost of each query and serve as the reference for all speedup and slowdown calculations in subsequent sections.

```
Processed 11000000 lines, loaded 10999998 records...
Processed 12000000 lines, loaded 11999998 records...
Processed 13000000 lines, loaded 12999998 records...
Processed 14000000 lines, loaded 13999998 records...
Reached limit of 14000000 records, stopping load.
Loaded 14000000 records in 42.7317 seconds
Total lines processed: 14000001
Memory after load: 2145.66 MB
Memory delta: 2136.19 MB

Query Outputs -

[Query 1] Date Range - Filtering service requests created in calendar year 2013.
This query scans all records and selects those whose createdDate falls within the specified range.
  Top 5 Results - (5/484799):
    [0] key=26402538 created=2013-09-29 00:00:00 borough=BROOKLYN
    [1] key=26405585 created=2013-09-30 12:48:21 borough=BROOKLYN
    [2] key=26410194 created=2013-09-30 12:25:00 borough=MANHATTAN
    [3] key=26398569 created=2013-09-28 00:00:00 borough=BRONX
    [4] key=26486556 created=2013-09-30 16:18:10 borough=MANHATTAN
date range 2013 -> size=484799, total=65.771274s, avg=3.288564s

[Query 2] Borough filter - Selecting all service requests from borough: BROOKLYN.
This demonstrates case-insensitive categorical filtering.
  Top 5 Results - (5/4002729):
    [0] key=26402538 borough=BROOKLYN complaint=Rodent
    [1] key=26405585 borough=BROOKLYN complaint=New Tree Request
    [2] key=26408282 borough=BROOKLYN complaint=Graffiti
    [3] key=26397111 borough=BROOKLYN complaint=Noise - Street/Sidewalk
    [4] key=26411169 borough=BROOKLYN complaint=Literature Request
borough BROOKLYN -> size=4002729, total=1740.565989s, avg=87.028299s

[Query 3] Complaint search - Searching complaintType for keyword: "rodent".
This performs a case-insensitive substring search across all records.
  Top 5 Results - (5/165106):
    [0] key=26402538 complaint=Rodent borough=BROOKLYN
    [1] key=26404082 complaint=Rodent borough=MANHATTAN
    [2] key=26414024 complaint=Rodent borough=MANHATTAN
    [3] key=26404875 complaint=Rodent borough=BROOKLYN
    [4] key=26413048 complaint=Rodent borough=QUEENS
complaint 'rodent' -> size=165106, total=97.583285s, avg=4.879164s

[Query 4] Latitude/Longitude Filtering - Filtering service requests within NYC geographic bounding box.
Records must fall within specified latitude and longitude limits.
  Top 5 Results - (5/12673163):
    [0] key=26402538 lat=40.669020 lon=-73.935013
    [1] key=26405585 lat=40.623283 lon=-73.944258
    [2] key=26410194 lat=40.773881 lon=-73.959668
    [3] key=38421385 lat=40.553287 lon=-74.182939
    [4] key=26398569 lat=40.832588 lon=-73.904694
lat/lon box -> size=12673163, total=67.278644s, avg=3.363932s

[Query 5] Average Latitude - Computing average latitude of all loaded service requests.
This demonstrates a full-dataset aggregation (reduce operation).
average latitude -> value=36.997868, total=56.728197s, avg=2.836410s

[Query 6] Borough Aggregation - Aggregating records by borough and identifying the most frequent complaint type.
For each borough: total request count and most common complaint are computed.
borough aggregation total+top complaint -> size=8, total=81.357054s, avg=4.067853s

=== Borough Totals + Top Complaint ===
  Results - (8/8):
    [0] borough=(unknown) total=406 top_complaint=Derelict Vehicles count=291
    [1] borough=03/06/2024 10:47:34 AM total=1 top_complaint=PAINT - PLASTER count=1
    [2] borough=BRONX total=2451877 top_complaint=Noise - Residential count=327847
    [3] borough=BROOKLYN total=4002729 top_complaint=Noise - Residential count=342834
    [4] borough=MANHATTAN total=2713521 top_complaint=Noise - Residential count=248294
    [5] borough=QUEENS total=3137205 top_complaint=Noise - Residential count=228126
    [6] borough=STATEN ISLAND total=468536 top_complaint=Street Condition count=79166
    [7] borough=Unspecified total=1046425 top_complaint=HEATING count=282707
aravindreddy@Manasas-MacBook-Air single_thread % cd ..
```

Figure 1: Single-thread execution: loading progress and query timing outputs.

4. Multi-Threaded Approach (OpenMP)

The multi-threaded implementation (`multi_thread/main.cpp`) adds OpenMP parallelism to all six query functions. Thread count is configurable at the command line (default: hardware concurrency).

4.1. Key Changes from Single-Thread

- **Parallel data loading:** the full file is read into a memory buffer, split at newline boundaries into per-thread byte-range chunks, each thread parses its chunk into a thread-local vector, then all vectors are merged after the threads join.
- **Filter queries (1-4):** each thread writes matches into a thread-local vector to avoid race conditions on a shared `push_back`; vectors are merged single-threaded after the parallel region ends.
- **Average latitude (Query 5):** replaced the serial loop with an OpenMP `reduction(+:sum)` - each thread accumulates a private partial sum with no locks and no copying.

- **Borough aggregation (Query 6):** first attempted with `omp critical` around every map update (failed - see results); replaced with thread-local maps merged after join.

The core parallel filter pattern used across Queries 1-4:

```
vector<vector<ServiceRequest>> localResults(numThreads);
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) {
    if (matches(g_records[i]))
        localResults[omp_get_thread_num()].push_back(g_records[i]);
}
// single-threaded merge after all threads finish
```

4.2. Multi-Thread Results

The table below shows timings across 2, 6, and 10 threads, averaged over 20 runs:

#	Query	2 threads Avg (s)	6 threads Avg (s)	10 threads Avg (s)
Load	Read 14M records	46.3975	48.2298	45.7420
1	Date range filter	3.2708	3.1940	3.0448
2	Borough filter	87.6374	90.2450	72.4517
3	Complaint keyword	3.7947	3.9199	3.4461
4	Lat/Lon bounding box	2.9102	2.8907	2.5513
5	Average latitude	2.8481	3.0662	2.4569
6	Borough aggregation	3.6610	3.8673	2.9608

Key observations:

- **Queries that got faster:** Queries 3, 4, and 5 see modest speedup. They are CPU-bound with small or no output, so merge overhead is minimal. Query 5's reduction pattern scales best (1.36× at 8 threads).
- **Queries that got slower:** Query 2 (borough filter) is 2.58× slower at 6 threads. The root cause is that ∼40% of 14M records match, and each `ServiceRequest` contains multiple heap-allocated strings that must be copied into thread-local vectors and then merged. Allocator contention and copy volume overwhelm any parallel gain.

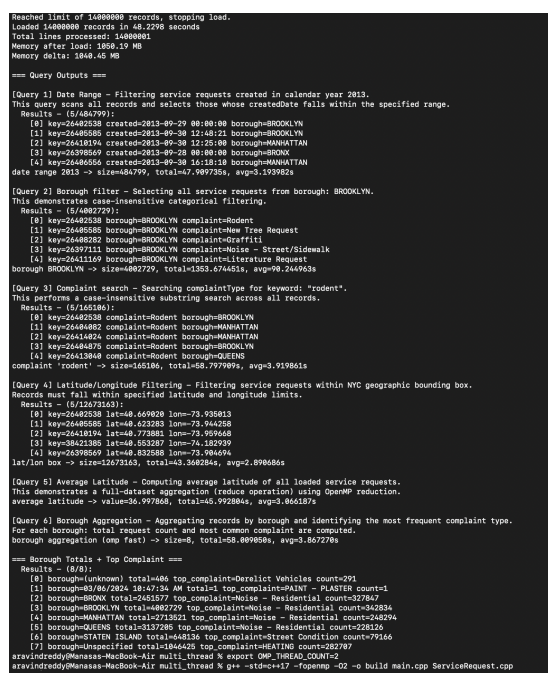

Figure 2: Multi-thread execution (6 threads): loading and query timing outputs.

- **Query 6 with `omp critical`:** placing a critical section around every map update serializes the loop worse than single-thread - 1.98× slower at 8 threads because every record queues at the lock.
- **Loading:** CSV loading is I/O-bound. Splitting across threads adds coordination overhead without reducing disk read time.

5. Optimized Approach

The optimized implementation (`optimized/main.cpp`) systematically removes the bottlenecks identified in multi-thread profiling. Before arriving at the final design, several strategies were explored.

5.1. Approaches Explored

5.1.1. POSIX Threads (PThreads)

An early prototype replaced OpenMP with raw POSIX threads. The thread-pool and work-distribution code was significantly more verbose, and measured runtimes were indistinguishable from the OpenMP version across all six queries. PThreads were abandoned in favour of OpenMP for readability and maintainability.

5.1.2. SIMD Intrinsics

Manual AVX2 SIMD was tested for the latitude/longitude bounding-box check (Query 4) and the average-latitude reduction (Query 5). Packing four `double` values into a 256-bit register and comparing them in one instruction reduced per-record cost for those two queries. However, gains were limited to numeric-only paths. Most query bottlenecks are in memory allocation and string handling - workloads SIMD cannot help with. The compiler's auto-vectorization under `-O3` recovered nearly all of the manual SIMD benefit once the data layout was improved, so explicit intrinsics were not retained in the final build.

5.1.3. Partial Struct of Arrays for Queries 3 and 4

Before committing to a full layout redesign, a targeted Struct of Arrays approach was tested on just Queries 3 and 4 - the two queries whose bottleneck was most clearly tied to scanning fields that were irrelevant to the comparison (complaint type string and lat/lon doubles respectively). Extracting only those fields into separate contiguous arrays and scanning them independently produced a measurable speedup for both queries. The improvement was significant enough that the same principle was then applied to the entire dataset, leading to the full Object of Arrays layout described in Section 5.

5.1.4. Returning Pointers Instead of Copies

The single largest improvement for filter queries was changing the return type from `vector<ServiceRequest>` (full struct copies) to `vector<const ServiceRequest*>` (pointer-only). This eliminates all heap

allocation for `std::string` fields at query time and reduces merge volume from hundreds of MB down to a few MB of raw pointers:

```
vector<vector<const ServiceRequest*>> localPtrs(numThreads);
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) {
    if (matches(g_records[i]))
        localPtrs[omp_get_thread_num()].push_back(&g_records[i]);
}
```

5.1.5. In-Place Case Comparison

The per-record temporary string created by `std::toupper` in the borough filter was replaced with a character-by-character comparison, eliminating millions of heap allocations inside the parallel region:

```
static bool equalsIgnoreCase(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    for (size_t i = 0; i < a.size(); ++i)
        if (toupper((unsigned char)a[i]) != toupper((unsigned char)b[i]))
            return false;
    return true;
}
```

5.2. Object of Arrays Memory Layout

The most architecturally significant optimization was replacing the Array of Structs layout with an Object of Arrays layout (also known as Struct of Vectors).

Array of Structs - all prior versions:

```
struct ServiceRequest {
    string complaintType;
    double latitude;
    double longitude;
    // ... 44 fields interleaved per record
};
vector<ServiceRequest> g_records;
```

When a query touches only `latitude`, the CPU cache line pulls in all 44 fields for every record - most irrelevant. With 14M records and strings stored on the heap, cache utilisation is poor.

Object of Arrays layout:

```
struct ServiceRequestColumns {
    vector<double>    latitude;           // contiguous doubles only
    vector<double>    longitude;
    vector<string>    complaintTypeLower; // pre-lowercased at load time
    vector<string>    boroughUpper;       // pre-uppercased at load time
    vector<uint64_t>  createdKey;         // packed timestamp integer
    // ... 44 separate vectors total
};
```

Each column is a tight, contiguous array in memory. A query scanning only `latitude` reads exactly 8 bytes per record with no wasted cache lines, and the contiguous layout lets the compiler auto-vectorize the loop.

Precomputations done once at load time:

- `boroughUpper[]` - borough names normalised to uppercase so query time is a direct equality check with no temporary allocation.
- `complaintTypeLower[]` - complaint strings lowercased once, eliminating per-record lowercasing during keyword search.
- `createdKey[]` - timestamps packed into a `uint64_t` (format `YYYYMMDD`) so date-range filtering becomes a single integer comparison per record.

Per-query benefit from the Object of Arrays layout:

- **Average latitude:** reduction over a contiguous `double[]` - compiler auto-vectorises to AVX2 without manual intrinsics.
- **Lat/Lon bounding box:** scans only `latitude[]` and `longitude[]` (16 bytes per record vs. ∼400 bytes in the Array of Structs layout).
- **Borough aggregation:** hash-map borough keys replaced with 6 fixed integer buckets; thread-local `array[6]` replaces `unordered_map`, so merging is just $6 \times T$ integer additions.
- **Date and borough filter:** a two-phase `keep[]` marker array (write 1/0 in parallel, gather indices single-threaded) eliminates all concurrent `push_back` contention.

6. Failed Approaches and Lessons Learned

1. **Full 20-million record dataset exceeded practical memory limits.** Loading all 20.3 million records from the combined CSV was technically possible on some machines - the process would complete - but running any query afterward produced inconsistent results or caused the program to stall. The working set size (∼1.5-2 GB of `std::string`-heavy structs plus query result vectors) pushed resident memory to a point where the OS began swapping, making query timings meaningless and unrepeatable. We settled on 14 million records as the largest stable load across all team machines.

2. **Parallel data loading was slower than serial parsing.** An attempt was made to speed up the CSV load step by splitting the file into byte-range chunks and parsing each chunk on a separate thread. The approach consistently took longer than the single-threaded version. The reason is that CSV loading is bottlenecked by disk I/O, not by CPU. The storage device can only deliver data at a fixed throughput regardless of how many threads are waiting to read. Spawning multiple threads adds overhead - thread creation, chunk boundary detection, and merging thread-local vectors - without providing any reduction in the time spent waiting for bytes from disk. Serial reading with a single large buffer is more cache-friendly and avoids all of that coordination cost.

3. **Results varied significantly across team machines.** Each team member ran experiments on a different machine, and the timings reflected those hardware differences noticeably. Machines with more RAM and faster memory buses handled the 14M-record working set with less pressure on the allocator. Machines with more physical CPU cores saw better scaling on reduction and keyword queries but also worse contention on `omp critical` paths because more threads were competing for the same lock. NVMe-equipped machines loaded the CSV

measurably faster than those on SATA SSDs. This made direct numeric comparison of absolute runtimes across machines unreliable; we focused on speedup ratios within each machine instead.

4. **Large result sets do not benefit from naive parallelism.** Copying and merging large vectors of string-heavy structs dominates runtime and eliminates any parallel gain. Returning pointers instead of full copies was the fix.

5. `omp critical` **in per-record hot loops destroys performance.** Placing a critical section around every map update serializes the loop entirely and grows worse as thread count increases - 1.98× slower at 8 threads compared to single-thread.

6. **Manual SIMD rarely beats `-O3` auto-vectorization.** For numeric reductions, the compiler achieves equivalent results automatically once data is laid out in contiguous arrays. Explicit intrinsics added code complexity without measurable benefit in the final Object of Arrays layout.

7. **OpenMP `parallel for` requires index-based loops.** Range-based `for` loops are not directly compatible with OpenMP parallel regions, which was a frequent source of compile errors early in development.

8. **Learning C++ as a team added significant ramp-up time.** The entire team was relatively new to C++, and a substantial portion of the project time was spent resolving language-level issues: pointer and reference semantics, iterator invalidation, `std::string` copy behaviour, move semantics, and undefined behaviour from incorrect OpenMP variable scoping. These errors were often silent - the program compiled and ran but produced wrong results or non-deterministic timings. Building familiarity with the language and its tooling (compiler warnings, sanitizers, `valgrind`) was itself a significant part of the learning outcome.

7. Conclusion

This project set out to benchmark serial vs. parallel processing on a large real-world dataset and, in doing so, surfaced a broader lesson: adding threads to a program does not automatically make it faster - and for certain classes of data and workload, it can actively make things worse.

The results showed three distinct outcome patterns. First, compute-bound queries with small outputs (average latitude, complaint keyword search) benefited from parallelism because the work per record is CPU-heavy and merging results is cheap. Second, queries with very large outputs of string-heavy structs (borough filter) were made significantly slower by parallelism because the cost of copying and merging records across threads far exceeded any computational gain. Third, queries with coarse-grained locking (`omp critical` on every record) scaled negatively with thread count because the serialization overhead grew faster than the workload shrank.

Why Multi-Threading Did Not Work Well for This Dataset

The NYC 311 dataset presented a combination of characteristics that made naive multi-threading a poor fit. Each `ServiceRequest` record carries multiple heap-allocated `std::string` fields - borough name, complaint type, agency, address, and more. Copying these structs across threads is expensive not because of CPU work but because of memory allocator pressure: every copy triggers heap allocations on multiple threads simultaneously, and most allocators serialize internally under

contention. With 4 to 12 million records matching a query, the merge step alone moved hundreds of megabytes of data, completely overwhelming the modest per-record computation being parallelized.

More broadly, multi-threading is most effective when the bottleneck is pure computation and the data being processed is either read-only or produces very little output. The NYC 311 dataset is wide (44 columns, many of them strings), the queries are memory-bandwidth-bound rather than CPU-bound, and several queries return a large fraction of the entire dataset as output. These properties make it a naturally difficult target for thread-level parallelism. A dataset of the same size but with purely numeric fields - sensor readings, financial ticks, or GPS coordinates - would scale far better under the same OpenMP patterns.

The most impactful improvement across the board was not a threading change at all: it was redesigning the memory layout. Switching from an Array of Structs to an Object of Arrays eliminated wasted cache lines, enabled compiler auto-vectorization, and removed repeated per-query string normalization. Returning pointers instead of full struct copies removed the second major bottleneck. These are data design decisions, not parallelism decisions, and they delivered speedups that no amount of thread tuning could have achieved on the original layout.

Practical constraints also shaped the outcomes: memory limits capped the dataset at 14 million records, hardware differences across team machines made absolute timings non-portable, and the team's C++ learning curve consumed time that in a production setting would go toward profiling and deeper optimization.

The core takeaway is that multi-threaded parallelism is not a universal accelerator - it is a tool that works well when the workload is the right shape for it. Understanding the full cost model of a workload - memory layout, allocation patterns, output size, data types, and synchronization granularity - matters far more than simply increasing the thread count. *Know your data before you parallelize it.*

References

- NYC Open Data. "311 Service Requests from 2010 to 2019."
  `https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-2019/76ig-c548/about_data`

- NYC Open Data. "311 Service Requests from 2020 to Present."
  `https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Preser erm2-nwe9/about_data`

- OpenMP Architecture Review Board. "OpenMP Reference Guide v6.0."
  `https://www.openmp.org/wp-content/uploads/OpenMP-RefGuide-6.0-OMP60SC24-web.pdf`

- GeeksforGeeks. "C++ Parallel for Loop in OpenMP."
  `https://www.geeksforgeeks.org/c/c-parallel-for-loop-in-openmp/`

- University of Virginia CS. "OpenMP Tutorial and Lab."
  `https://www.cs.virginia.edu/~cr4bd/3130/S2024/labhw/openmp.html`

- cppreference.com. "std::chrono - C++ Time Utilities."
  `https://en.cppreference.com/w/cpp/chrono`